

Overall Architecture:

Client / # The frontend

TestAudit.usx essentially provides debugging and raw data for testing purposes.

When button pressed, it sends POST request to server, then polls every 2 seconds for a GET to `/api/audits/:id`. Then renders the minimal JSON response.

Server /

Receives the POST request from the client, then sets the status to "pending" in PostgreSQL, then adds the job to Redis Queue.

After worker handles the job...

Retrieves the audit table, metrics table, and suggestions table. Then sends that information to the client upon a GET request.

Worker /

polls the Redis Queue for jobs.

Upon picking up a job, it sets the status to "running". Then launches Puppeteer and Lighthouse to audit metrics.

Then saves the metrics and AI-based suggestions to the PostgreSQL database tables, and set status to Complete.